14 — Eğitilebilir Modeller & MLOps: Derinlemesine (Faz 7)

> **Felsefe:** Aynı kod, farklı şirketler. Her şirketin kendi verisi var, kendi modelini eğitir. Modeli versiyonla, A/B test et, geriye al. Yöneticiler “Excel yükle, model güncellesin” yapabilsin.

---

1. Niye eğitilebilir model?

1.1 Soru

> “Tek bir model her şirkete uyar mı?”

Hayır. Her şirketin: - Sektörü farklı (plastik enjeksiyon, tekstil, gıda) - Bayi yapısı farklı (5 büyük bayi vs 200 küçük bayi) - Mevsimselliği farklı (yaz/kış, bayram öncesi) - Ürün karması farklı (10 ürün vs 1000 ürün)

Tek bir “MatPortal modeli” satarsak, her müşteride %30-40 hata vermeye mahkum.

1.2 Çözüm

Paylaşılan kod, müşteriye özel veri & model.

```
matportal-core (paketlenmiş kod)
├── tahmin/algoritmalar/      ← naive, prophet, xgboost, lstm (ortak)
├── ozellik_muhendisligi/     ← feature engineering modülleri (ortak)
└── deploy/

paspas-deploy/                 ← Promats için
├── data/                      ← Promats verisi
├── modeller/                  ← Promats için eğitilmiş modeller
└── konfig.json                ← sektör=plastik, mevsim=yıllık vs.

ornek-tekstil-deploy/          ← başka müşteri
├── data/
├── modeller/
└── konfig.json                ← sektör=tekstil, mevsim=sezonsal
```

---

2. MLOps altyapısı

2.1 Sorumluluk dağılımı

Sorumluluk	Kim	Araç
Veri toplama	Paspas backend (cron)	Drizzle ORM
Veri kalite kontrol	`data_quality` modülü	Custom + great-expectations
Feature engineering	`feature_pipeline` modülü	Pandas + numpy
Model eğitim	`training_service`	Python (Bun → spawn)
Model versiyonlama	DB + S3	`tahmin_modelleri` tablosu
Model deploy	API üzerinden yükle	FastAPI (Python serving)
A/B test	`ab_test_router`	Custom
İzleme	Grafana + custom metrik	Prometheus
Geriye alma (rollback)	UI button	DB transaction

2.2 Hibrit stack — Bun + Python

Paspas backend Bun/TypeScript. ML kütüphaneleri (scikit-learn, prophet, xgboost) Python ekosisteminde. Köprü:

```
┌──────────────────────────────────────────┐
│  Paspas Backend (Bun)                    │
│  - REST API                              │
│  - DB CRUD                               │
│  - UI integrasyon                        │
└──────────────┬───────────────────────────┘
               ↓ HTTP / queue
┌──────────────────────────────────────────┐
│  ML Service (Python FastAPI)             │
│  - /train  (POST)                        │
│  - /predict (POST)                       │
│  - /evaluate (POST)                      │
│  - /model/{id} (GET, DELETE)             │
└──────────────┬───────────────────────────┘
               ↓
┌──────────────────────────────────────────┐
│  MLflow (deneme takibi)                  │
│  + S3 (model artifacts)                  │
└──────────────────────────────────────────┘
```

2.3 ML Service — özellik listesi

- **/train**: model eğitimi başlat. Asenkron. İş kuyruğuna gider, BullMQ/Celery.
- **/predict**: production tahmin. Düşük latency (<100ms hedef).
- **/evaluate**: backtest. Geçmiş veride model performansı.
- **/model/{id}/promote**: A → B'ye geçir (champion).
- **/model/{id}/rollback**: önceki stabile dön.

---

3. Model versiyonlama

3.1 Versiyonlama şeması

Her model bir **artifact**:

```
matportal-models/
├── promats/
│   ├── tahmin/
│   │   ├── prophet-bayi-042-urun-star-siyah/
│   │   │   ├── v1.0/  # ilk eğitilmiş
│   │   │   ├── v1.1/  # haftalık güncelleme
│   │   │   ├── v1.2/  # 2 hafta önce
│   │   │   └── v1.3/  # şu anki champion
│   │   └── xgboost-genel-tum-bayi/
│   └── churn/
│       └── xgboost-bayi-churn-60gun/
```

3.2 Versiyon metadata

```
CREATE TABLE model_versiyonlari (
  id char(36) PK,
  model_ailesi varchar(128),     -- 'prophet-bayi-042-urun-star-siyah'
  versiyon varchar(32),           -- 'v1.3'
  egitim_baslangic datetime,
  egitim_bitis datetime,
  egitim_kayit_sayisi int,
  hyperparametreler JSON,
  metrikler JSON,                 -- { mape, rmse, bias, coverage }
  artifact_yolu text,             -- s3://matportal-models/.../v1.3/
  durum ENUM('egitiliyor','hazir','champion','retired','hatali'),
  champion_olma_tarihi datetime,
  retired_tarih datetime,
  egiten_user_id char(36),
  notlar text
);
```

3.3 Champion/Challenger pattern

- **Champion**: production'a veriyor. Her tahmin bunu kullanıyor.
- **Challenger**: yeni eğitim. Production trafiğinin **%5'i** ile test ediliyor.
- 7 gün sonra: challenger'ın MAPE'si champion'dan iyi ise → promote.
- Promote edilince eski champion **retired** olur ama silinmez (rollback için).

---

4. A/B test

4.1 Trafik bölme

```
Bayi X için tahmin isteği geldi
  ├─ %95 → champion v1.3 → tahmin döner
  └─ %5  → challenger v1.4 → tahmin döner (sessiz, log)

Her iki tahminin gerçekleşen değerle karşılaştırması yapılır.
```

4.2 İstatistiksel anlamlılık

```
def ab_test_anlamli_mi(
  champion_mape: list[float],
  challenger_mape: list[float],
  alfa: float = 0.05
) -> bool:
  from scipy import stats
  t_stat, p_value = stats.ttest_ind(champion_mape, challenger_mape)
  return p_value < alfa and challenger_mape.mean() < champion_mape.mean()
```

4.3 Karar kuralı

Kriter	Karar
Challenger MAPE > %20 daha iyi + p < 0.05	**Promote** (challenger → champion)
Eşit veya kötü + p >= 0.05	Reddedilir, retire edilir
Sınır durumda + örnek sayısı az	Test devam eder, 7 gün ek

---

5. Excel upload UX

5.1 İki senaryo

Senaryo A — Tarihsel veri ekleme

Yönetici geçmiş 2 yılın siparişlerini başka sistemden ihraç etmiş, Excel olarak elinde.

Senaryo B — Yeni özellik ekleme

Yönetici “kampanyalarımız” Excel’i tutuyor — kampanyaları model’e tanıtmak istiyor.

5.2 Akış

```
1. /admin/tahmin-motoru/veri/yukle
2. Şablon indir butonu (siparis_v1.xlsx, kampanya_v1.xlsx, ham_madde_v1.xlsx)
3. Drag-drop dosya yükle
4. Sistem doğrulama:
   - Kolon eşleştirme (önce otomatik, sonra manuel onay)
   - Veri tipi kontrolü
   - Referans bütünlüğü (bayi_kod, urun_kod var mı?)
   - Outlier tespit
5. Preview: "150 satır okundu, 142 geçerli, 8 hata"
6. "Hatalı satırları indir" + "Geçerlileri kaydet"
7. DB'ye yazılır → "Modeli yeniden eğit" butonu aktifleşir
8. Buton tıklanır → arka plan eğitim başlar
9. Tamamlanma: bildirim + email + "yeni model X performans Y" raporu
```

5.3 Kolon eşleştirme UI

```
Yüklenen dosya kolonları:    →    Sistem kolonları
"Tarih"                       →    [tarih ▼]            ✓ otomatik tespit
"Bayi Adı"                    →    [bayi_kod ▼]         ⚠ "ad" mı "kod" mu?
"Ürün Kodu"                   →    [urun_kod ▼]         ✓
"Adet"                        →    [adet ▼]             ✓
"Liste Fiyatı"                →    [birim_fiyat ▼]      ✓
"İskonto %"                   →    [iskonto_yuzde ▼]    ✓ (5 = %5)
"Kampanya"                    →    [kampanya_kodu ▼]    ✓
"Açıklama"                    →    [notlar ▼]           ✓
```

5.4 Hata düzeltme inline

```
Satır 42: bayi_kod "BAY-XYZ" bulunamadı
  ▼ Çözüm seçin:
    [ Yeni bayi olarak oluştur ]
    [ Mevcut bayiye eşle ]
    [ Bu satırı atla ]
```

6. Çok değişkenli model — multivariate

6.1 Tek değişkenli vs çok değişkenli

Tek değişkenli (univariate): Sadece geçmiş sipariş adedinden tahmin. - Naive, ARIMA, basit Prophet

Çok değişkenli (multivariate): Sipariş + ek faktörler. - Prophet (regressors), XGBoost, LSTM

6.2 Eklenecek özellikler

Kategori	Özellik	Veri kaynağı
Makro	USD/TRY kuru	TCMB API (geçmiş 5 yıl)
	EUR/TRY kuru	TCMB API
	Enflasyon (TÜFE)	TÜİK
	Petrol fiyatı (Brent)	Yahoo Finance
Takvim	Resmi tatil	Sabit JSON
	Dini tatil	Sabit JSON
	Hafta sonu	Hesaplama
	Ay başı/sonu	Hesaplama
Hava	Sıcaklık (sezonsal sektör için)	OpenWeather Historical
	Yağış	OpenWeather
Kampanya	Aktif kampanya	`kampanyalar` tablosu
	İskonto oranı	`siparis_kalemleri.iskonto_yuzde`
Stok	Bayi stoku (tahmini)	Web kazıma + manuel
Bayi profil	Sektör (one-hot)	`musteriler.sektor`
	Ölçek (küçük/orta/büyük)	`musteriler.calisan_sayisi`
	Bölge	`musteriler.il`

6.3 Feature engineering pipeline

```
def ozellik_uret(bayi_id, donem_baslangic, donem_bitis):
    df = get_siparis_history(bayi_id, donem_baslangic - 365, donem_bitis)

    # Lag features
    df['siparis_t_minus_1'] = df['adet'].shift(1)
    df['siparis_t_minus_7'] = df['adet'].shift(7)
    df['siparis_t_minus_30'] = df['adet'].shift(30)

    # Rolling features
    df['7gun_ortalama'] = df['adet'].rolling(7).mean()
    df['30gun_ortalama'] = df['adet'].rolling(30).mean()
    df['7gun_std'] = df['adet'].rolling(7).std()

    # Calendar features
    df['ay'] = df.index.month
    df['hafta'] = df.index.isocalendar().week
    df['gun_ay_ici'] = df.index.day
    df['hafta_gunu'] = df.index.dayofweek
    df['tatil_mi'] = df.index.isin(turk_tatilleri)
    df['ramazan_mi'] = df.index.isin(ramazan_donemi)

    # Macro features (left join)
    df = df.merge(usd_try_history, left_index=True, right_index=True, how='left')
    df = df.merge(weather_history, left_index=True, right_index=True, how='left')

    # Bayi profili (sabit)
    bayi_profile = get_bayi_profile(bayi_id)
    for col in ['sektor', 'olcek', 'bolge']:
        df[col] = bayi_profile[col]

    return df
```

---

7. Active learning — kullanıcı düzeltmesinden öğrenme

7.1 Senaryo

```
SİSTEM: "Önümüzdeki 30 gün BAY-042 → 250 adet tahmin"
YÖNETİCİ: "Hayır, geçen ayın aynısı 180 olacak. Pazar zayıf."
```

Bu **değerli sinyal**. Yönetici bilgi sahibi (saha sezgisi). Modele geri besleme yapılır:

```
CREATE TABLE tahmin_geri_besleme (
  id char(36) PK,
  tahmin_id char(36),
  user_id char(36),
  duzeltilen_deger decimal(12,2),
  sebep text,                  -- "pazar zayıf", "müşteri kapanış"
  guven_dusuk_mu tinyint,      -- yönetici "emin değilim" işaretledi mi?
  geri_besleme_tarihi datetime
);
```

7.2 Modele entegrasyon

3 strateji:

A — Online learning

Düzeltme anında modele yedirilir. **Risk:** Yanlış düzeltme modeli bozar.

B — Batch learning + ağırlık

Hafta sonunda toplu yeniden eğitim. Düzeltme verisi normal veriden **2× ağırlık** alır.

C — Calibration layer

Model olduğu gibi kalır, üstüne “calibration” katmanı eklenir.

```
final_tahmin = ham_model_tahmin × (1 + calibration_factor)
calibration_factor = ortalama (yönetici_düzeltmeleri / ham_tahminler)
```

Önerim: B + C kombinasyonu. Ana model toplu öğrenir, calibration anlık esnetir.

7.3 Yönetici güvenliği

- Tüm düzeltmeler **audit log**.
- Aşırı düzeltme yapan kullanıcı **flag** (model bozulmasın).
- Çakışan düzeltme (2 yönetici farklı düzeltme) → ortalama veya sonuncu.

---

8. Açıklanabilirlik — explainability

8.1 Niye?

Yönetici “neden 250 dedin?” sorduğunda **anlamlı cevap** alabilmeli. Yoksa sistem siyah kutu olur, güven azalır.

8.2 Yöntemler

Yöntem	Hangi modelde	Avantaj
Feature importance	Tüm tree-based (XGBoost, LGBM, RF)	Hızlı, basit
SHAP values	Herhangi model	Tahmin başına detaylı
Permutation importance	Herhangi model	Yavaş ama tutarlı
Partial Dependence Plot	Tüm modeller	“Şu özellik artarsa tahmin nasıl değişir?”
LIME	Herhangi model	Tek tahmini açıklamak için

8.3 UI’da gösterim

```
Tahmin: 250 adet
↓ neden bu tahmin?
┌──────────────────────────────────────────┐
│ Geçen yıl aynı ay      ▇▇▇▇▇▇▇  +40      │
│ Son 30 gün ortalama    ▇▇▇▇▇    +30      │
│ Mayıs ay-içi yüksek    ▇▇▇      +20      │
│ Tatil yakın             ▇▇      +15      │
│ USD/TRY artış          ▇▇       -10      │
│ Bayi son 30 gün durgun ▇▇       -15      │
│ Kampanya aktif değil   ▇         -10     │
└──────────────────────────────────────────┘
Net: 250 (220-285 güven aralığı %95)
```

8.4 Doğal dil açıklama (LLM)

```
PROMPT: "Şu SHAP değerlerini doğal dile çevir:
{geçen_yıl: +40, son_30gün: +30, mayıs: +20, ...}
Yöneticiye anlamlı 2-3 cümle açıklama ver."

ÇIKTI: "Geçen yıl Mayıs ayında bu bayi 240 adet sipariş vermişti.
Son 30 günde aktivite normalin %15 üstünde, ama kur artışı küçük
düşüş etkisi yapıyor. Bayinin geçen 30 gün durgunluğu hesaba katılmış."
```

---

9. Model deployment — canary

9.1 Yeni model çıkışı

1. Eğitim tamamlandı (offline)
2. Backtest geçti (MAPE eşik altı)
3. Staging'de 24 saat çalıştı (canary)
4. %5 production trafiğine açıldı (challenger)
5. 7 gün izlendi
6. Champion'a göre iyi → promote
7. Eski champion 30 gün retired tut
8. 30 gün hata yoksa silinebilir (S3 maliyet)

9.2 Hata durumunda

- Auto-rollback eşiği: gerçek MAPE > %50 olursa **otomatik** önceki stabil sürüme dön
- Yöneticiye email + dashboard kırmızı

---

10. Çok-müşteri (multi-tenant) izolasyon

10.1 Senaryo

Promats için MatPortal canlı. Sonra başka bir plastik fabrikasına da satıldı. **İki müşterinin verisi karışmamalı.**

10.2 Yaklaşım — separate deployment

Tek codebase, **iki ayrı VPS / iki ayrı DB**.

```
matportal-promats.com.tr   ← Promats deployment
matportal-firma2.com.tr    ← İkinci müşteri
```

- DB ayrı: `promats_erp`, `firma2_erp`
- Model storage ayrı: `s3://matportal/promats/`, `s3://matportal/firma2/`
- LLM API key ayrı (her müşteri kendi limit'i)
- Konfig farklı (sektör, ölçek, mevsimsellik)

10.3 Federated learning (uzun vadeli)

Çok müşteri olduğunda **anonim** federated model: - Her müşterinin lokal modeli kendi verisinde eğitilir - Sadece **ağırlıklar** merkez sunucuya gönderilir (veri gönderilmez) - Merkezi "meta-model" oluşur - Yeni müşterinin "ilk gün" performansı meta-model sayesinde iyi başlar

Faz 7'de değil, Faz 11+ vizyonu.

---

11. Maliyet ve performans

11.1 Eğitim maliyeti

Model türü	Eğitim süresi	RAM	CPU
Naive	<1 sn	100MB	1 vCPU
Mevsimsel	1 sn	100MB	1 vCPU
Lineer regresyon	5-10 sn	200MB	1 vCPU
Prophet	30-60 sn / model	500MB	1-2 vCPU
XGBoost	1-3 dk / model	1GB	2-4 vCPU
LSTM	5-30 dk / model	2-4GB	GPU önerilir

500 bayi-ürün modeli × 1 dk Prophet = 8 saat. Pazartesi 03:00 cron'da bitsin diye **paralel** eğitim (8 worker × 1 saat).

11.2 Tahmin (inference) maliyeti

- Prophet: 50-100ms/tahmin
- XGBoost: 5-10ms/tahmin
- Cache: 1 saatlik tahmin sonucu Redis'te → tekrar istenirse anlık

11.3 Storage

- Model artifact: 1-50MB / model
- 500 model × 30MB = 15GB
- 12 versiyon × 15GB = 180GB
- S3 ~$4/ay

---

12. DB şeması — ek tablolar

```
CREATE TABLE feature_pipeline_kayitlari (
  id char(36) PK,
  pipeline_versiyonu varchar(32),
  girdi_tablo varchar(64),
  feature_count int,
  null_oran decimal(4,2),
  outlier_count int,
  calistirma_zamani datetime,
  basarili tinyint
);

CREATE TABLE ab_test_calistirma (
  id char(36) PK,
  champion_model_id char(36),
  challenger_model_id char(36),
  baslangic datetime,
  bitis datetime,
  champion_mape decimal(8,4),
  challenger_mape decimal(8,4),
  p_value decimal(10,8),
  istatistiksel_anlamli tinyint,
  sonuc ENUM('beklemede','promote','retire','sürer')
);

CREATE TABLE excel_yukleme_gecmisi (
  id char(36) PK,
  user_id char(36),
  dosya_tipi ENUM('siparis','kampanya','ham_madde','musteri','urun'),
  dosya_adi varchar(255),
  satir_sayisi int,
  basarili int,
  hata int,
  durum ENUM('isleniyor','tamamlandi','hatali'),
  hata_log JSON,
  yeniden_egitim_tetikledi tinyint,
  yuklendi_at datetime
);
```

13. UI — `/admin/ml-laboratuvari`

13.1 Dashboard

- **Aktif modeller**: liste (model ailesi, versiyon, MAPE, son eğitim)
- **Eğitim kuyruğu**: "şu anda 5 model eğitiliyor"
- **A/B testler**: aktif test listesi, sonuç bekleniyor
- **Veri sağlığı**: outlier %, eksik %, duplicate %

13.2 Model detay

- Eğitim geçmişi (versiyonlar tablosu)
- Performance trend grafiği
- Feature importance bar chart
- "Yeniden eğit" butonu
- "Rollback" butonu
- Hyperparameter geçmişi

13.3 Yönetici geri besleme

- Bayi-ürün tahmini gösterilir
- "Düzelt" butonu
- Düzeltme nedeni metin alanı
- "Modeli yenile" butonu (calibration sürer)

---

14. Açık karar noktaları (Faz 7)

1. **Python ML service:** ayrı container mı, Bun process mi spawn? (Önerim: ayrı container, FastAPI)
2. **MLflow vs custom:** açık kaynak çözüm mi yazıyoruz mu? (Önerim: MLflow başla, gerekirse custom)
3. **Federated learning:** Faz 7'de mi, sonraki faz mı? (Önerim: sonraki)

4. **Excel upload kolon mapping:** otomatik tespit ne kadar gelişmiş? (Önerim: önce manuel, AI sonra)
5. **GPU gerek olur mu:** LSTM için VPS yetersiz, cloud GPU mu? (Önerim: ihtiyaç doğunca AWS Sagemaker)
6. **Calibration vs full retrain:** ne sıklıkla kalibrasyon? (Önerim: günlük inkremental, haftalık tam)
7. **Audit retention:** model versiyon kaç yıl saklanır? (Önerim: 3 yıl + retired sonra silinir)

---

15. Faz 7 çıktı listesi

✅ Python ML service (FastAPI) ayağa kalktı ✅ MLflow + S3 model storage ✅ Excel upload UX (kolon mapping + preview + hata düzeltme) ✅ Multivariate feature pipeline ✅ XGBoost + Prophet modelleri çok değişkenli eğitilebiliyor ✅ A/B test mekanizması (champion/challenger) ✅ Active learning (yönetici düzeltmesi → calibration) ✅ Açıklanabilirlik (SHAP UI + LLM doğal dil) ✅ Auto-rollback (MAPE > %50 → eski modele dön) ✅ Multi-deployment ready (env-based config) ✅ Audit log: her eğitim, her tahmin, her geri besleme